

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	M.HASHETHA	Reg. No.:	192521162
Date:	18.08.2026	Duration:	60 minutes

Code of Conduct

I M.HASHETHA(192521162) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.HASHETHA

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

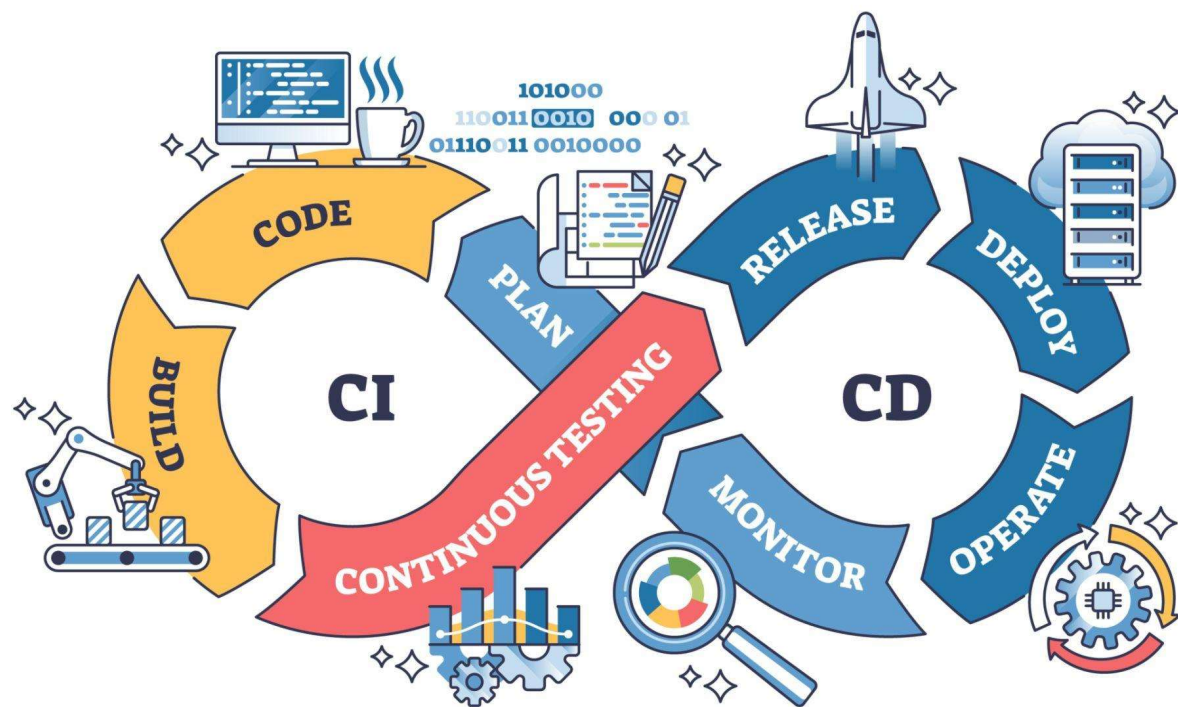
Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

AI-Powered Smart Court Case Management and Judicial Analytics Platform

Problem Statement:

Courts often experience delays in case processing due to manual documentation, inefficient scheduling, and limited analytical tools. Existing systems provide minimal support for case prioritization and judicial workload management. An AI-powered judicial platform should automate document classification, schedule hearings intelligently, analyse case trends, predict case durations, and provide dashboards for judicial administration. The system should improve court efficiency while ensuring transparency and accessibility.



Question 1: Analyse the Project Requirements and Identify the CI/CD Needs of the Assigned Problem Statement

Answer:

The **AI-Powered Smart Court Case Management and Judicial Analytics Platform** is a software system that uses AI, Natural Language Processing, cloud computing, and analytics to digitize court case management, automate legal document classification, optimize hearing schedules, predict case completion timelines, analyze judicial workload, and generate judicial analytics. Since the system handles confidential legal information and contains AI-based functionality, a reliable and secure CI/CD pipeline is required.



The main CI/CD needs of the proposed system are as follows:

1. **Source Code Management:** All application source code, AI/NLP modules, database scripts, configuration files, and infrastructure files should be maintained using a version control system such as Git.
2. **Continuous Integration:** Whenever developers commit or push code, the pipeline should automatically compile/build the application and verify that the newly integrated code does not introduce errors.
3. **Automated Testing:** Unit tests, integration tests, API tests, security tests, and selected end-to-end tests should execute automatically during the pipeline.
4. **Code Quality Analysis:** The source code should be automatically analyzed for coding errors, vulnerabilities, duplicated code, and maintainability issues before deployment.
5. **AI Model Validation:** Changes to AI/NLP components should undergo automated validation to ensure that document classification and prediction functionality continues to meet predefined accuracy and quality requirements.
6. **Secure Packaging:** After successful testing and quality checks, the application should be packaged into a deployable artifact or container image.
7. **Environment Management:** Separate **development, testing/staging, and production environments** should be maintained to prevent untested changes from directly reaching the production system.
8. **Security Integration:** Security scanning should be included throughout the pipeline to identify vulnerable dependencies, insecure configurations, exposed secrets, and application security issues.

9. **Automated Deployment:** Approved builds should be automatically deployed to the appropriate environment. Production deployment should require an appropriate approval or quality gate because the system handles sensitive judicial information.
10. **Monitoring and Notifications:** The pipeline should provide build and deployment status notifications and support application monitoring after deployment.
11. **Rollback and Recovery:** If a production deployment causes errors or unacceptable performance, the system should support quick rollback to the previously stable version.
12. **Auditability and Traceability:** All code changes, test results, approvals, deployments, and rollbacks should be recorded to support accountability and transparency.

Therefore, the CI/CD pipeline should provide **continuous integration, automated testing, quality assurance, security validation, controlled deployment, monitoring, and reliable rollback mechanisms**. This approach helps ensure that the judicial platform can be updated frequently while maintaining **security, reliability, quality, and ethical responsibility**.

Question 2: Design the CI/CD Pipeline Workflow Showing Stages Such as Source Code Management, Build, Testing, Code Quality Analysis, Packaging, and Deployment

Answer:

The CI/CD pipeline for the **AI-Powered Smart Court Case Management and Judicial Analytics Platform** is designed to automate the process of integrating source code, building the application, performing automated testing, checking code quality and security, packaging the application, and deploying it into different environments.

The proposed CI/CD workflow consists of the following stages:

1. Source Code Management:

Developers maintain the application source code, AI/NLP modules, database scripts, configuration files, and test cases in a Git-based repository. Every code change is committed and pushed to the repository, which triggers the CI/CD pipeline.

2. Build Stage:

The pipeline retrieves the latest source code and automatically builds the application. Dependencies are installed and compilation or build errors are checked. A failed build stops the pipeline from proceeding further.

3. Automated Testing Stage:

Automated unit tests, integration tests, API tests, and selected end-to-end tests are executed. The

pipeline verifies important functions such as user authentication, case registration, document processing, hearing scheduling, and analytics.

4. Code Quality and Security Analysis:

The source code is analyzed using automated quality and security tools. The pipeline checks for coding issues, vulnerabilities, duplicated code, insecure dependencies, and other quality problems. Critical issues should prevent further deployment.

5. AI/NLP Validation:

Changes to the AI-based document classification and prediction modules are validated using predefined test datasets. The pipeline checks whether the AI functionality continues to meet the required accuracy and quality thresholds.

6. Packaging:

After successful testing and quality checks, the application is packaged as a deployable artifact or container image. A unique version or build number is assigned to maintain traceability.

7. Development Deployment:

The successfully packaged application is automatically deployed to the development environment for further verification by developers.

8. Staging/Testing Deployment:

After successful development validation, the application is deployed to the staging environment. Additional integration, security, performance, and acceptance testing can be performed in an environment similar to production.

9. Production Deployment:

After all required quality gates are passed, the approved version is deployed to the production environment. Because the platform handles confidential judicial information, production deployment should include appropriate authorization or approval.

10. Monitoring and Rollback:

After deployment, the application is continuously monitored for errors, performance problems, and availability issues. If a serious problem occurs, the pipeline can automatically or manually roll back to the previous stable version.

CI/CD Workflow

Developer → Git Repository → Build → Automated Tests → Code Quality & Security → AI/NLP Validation → Package → Development → Staging → Approval → Production → Monitoring →

Rollback if Required

This workflow provides a controlled and automated path from **code commit to production deployment**. It reduces manual errors, improves software quality, enables faster releases, and ensures that security and testing are integrated throughout the development lifecycle.

Question 3: Identify Suitable Tools and Technologies for Each Stage of the CI/CD Pipeline and Justify Their Selection

Answer:

The selection of suitable CI/CD tools is important for automating development, testing, security, deployment, and monitoring of the **AI-Powered Smart Court Case Management and Judicial Analytics Platform**. The following tools are selected based on their reliability, ease of integration, automation capabilities, and suitability for a modern software development environment.

Pipeline Stage	Tool / Technology	Purpose and Justification
Source Code Management	Git + GitHub	Git provides version control, while GitHub provides repository management, collaboration, branch management, and pull requests.
CI/CD Automation	GitHub Actions	It automatically triggers builds, tests, security checks, and deployments whenever code is committed or a pull request is created.
Application Build	Maven / npm	Maven can manage Java-based applications, while npm can manage Node.js dependencies and builds. They automate dependency management and application packaging.
Unit Testing	JUnit / Jest	These frameworks automatically verify individual application components and help detect defects early.
API Testing	Postman / Newman	They can test APIs used for case management, authentication, document processing, and analytics.
End-to-End Testing	Selenium	Selenium can automate browser-based testing of important user workflows.

Pipeline Stage	Tool / Technology	Purpose and Justification
Code Quality	SonarQube	It identifies bugs, code smells, duplicated code, and maintainability issues, improving overall code quality.
Security Scanning	OWASP Dependency-Check	It identifies known vulnerabilities in third-party software dependencies and helps reduce security risks.
Containerization	Docker	Docker packages the application and its dependencies into a consistent container, reducing environment-related deployment problems.
Container Registry	GitHub Container Registry	It provides a location to securely store and manage versioned Docker images.
Deployment	Kubernetes	Kubernetes can manage containerized applications and support scalable and reliable deployment.
Cloud Platform	AWS / Azure	Cloud infrastructure provides scalable computing, storage, networking, and monitoring services for the judicial platform.
Monitoring	Prometheus + Grafana	Prometheus collects system metrics, while Grafana provides dashboards for monitoring application performance and availability.
Notifications	Email / Microsoft Teams	Developers and administrators can receive notifications about build failures, successful deployments, security issues, and pipeline status.

Justification

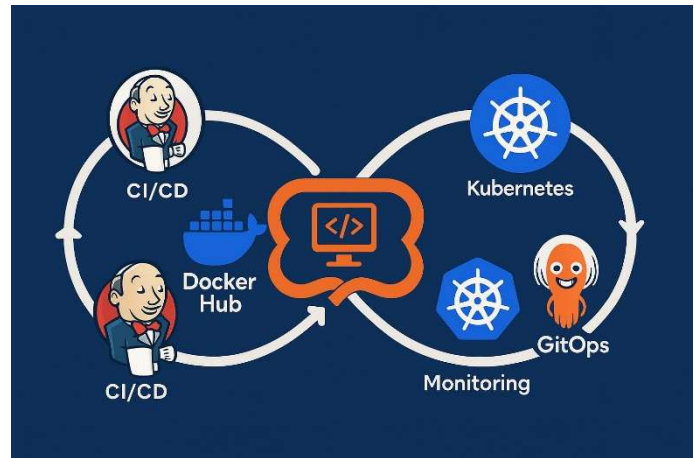
Git and GitHub provide reliable source-code version control and collaboration. GitHub Actions is suitable for implementing the CI/CD workflow because it integrates directly with the source repository and can automatically execute pipeline jobs.

JUnit/Jest, Postman, and Selenium provide multiple levels of automated testing, ensuring that application functionality, APIs, and important user workflows are verified. SonarQube and OWASP Dependency-Check integrate quality and security checks into the pipeline.

Docker and Kubernetes provide consistent, scalable, and reliable deployment. Cloud platforms such as AWS or Azure can provide the infrastructure required to operate the system.

Finally, Prometheus and Grafana support continuous monitoring after deployment.

Therefore, this combination of tools provides an integrated CI/CD environment covering source control, building, testing, quality analysis, security, packaging, deployment, monitoring, and notification, making it suitable for the proposed AI-powered judicial platform.



Question 4: Define Appropriate Automated Testing Strategies to be Integrated into the CI/CD Pipeline

Answer:

Automated testing is an important part of the CI/CD pipeline for the **AI-Powered Smart Court Case Management and Judicial Analytics Platform**. It ensures that every code change is tested automatically before it reaches the production environment. The testing strategy should cover functionality, integration, security, performance, and AI-related components.

1. Unit Testing

Unit tests verify individual functions and modules of the application. For example, functions related to user authentication, case registration, case searching, hearing scheduling, and report generation should be tested independently. **JUnit or Jest** can be used for automated unit testing.

2. Integration Testing

Integration testing verifies whether different components work correctly together. For example, the case management module should communicate correctly with the database, authentication system, document management module, and analytics module.

3. API Testing

The platform will use APIs for communication between different services. Automated API tests using **Postman/Newman** can verify authentication APIs, case management APIs, document processing APIs, scheduling APIs, and analytics APIs.

4. End-to-End Testing

End-to-end testing verifies complete user workflows. For example:

Login → Register Case → Upload Document → AI Classification → Schedule Hearing → Generate Report

Tools such as **Selenium** can automate important browser-based workflows.

5. Security Testing

Security tests should identify vulnerabilities in the application and its dependencies. Automated security scanning should check for insecure dependencies, exposed secrets, authentication weaknesses, and common application vulnerabilities. **OWASP Dependency-Check** can be integrated into the pipeline.

6. Code Quality Testing

Every code change should undergo automated code-quality analysis using **SonarQube**. The pipeline should detect bugs, code smells, duplicated code, and maintainability issues. Critical quality failures should stop the pipeline.

7. AI/NLP Model Testing

The AI-based document classification and case prediction modules should be tested using predefined datasets. The pipeline should verify classification accuracy and other predefined performance measures. Model changes should not be accepted if they cause performance to fall below the defined threshold.

8. Performance Testing

Performance tests should verify that the system can handle multiple users, large numbers of court cases, document uploads, and analytics requests. The tests should identify slow response times and resource-related problems.

9. Regression Testing

Previously tested features should be automatically tested whenever new code is introduced. This ensures that new changes do not break existing functionality.

Automated Testing Flow

Code Commit → Build → Unit Test → Integration Test → API Test → Security Scan → Code Quality Check → AI/NLP Validation → End-to-End Test → Package

Conclusion

The proposed automated testing strategy provides multiple levels of verification throughout the CI/CD pipeline. By integrating **unit, integration, API, end-to-end, security, code-quality, AI/NLP,**

performance, and regression testing, defects can be detected early and only stable, secure, and high-quality builds can proceed toward deployment. This improves the **reliability, security, maintainability, and overall quality** of the judicial platform.

Question 5: Design the Deployment Strategy, Including Development, Testing/Staging, and Production Environments

Answer:

The deployment strategy for the **AI-Powered Smart Court Case Management and Judicial Analytics Platform** should use separate environments to ensure that new software changes are properly developed, tested, approved, and deployed before reaching actual users. This reduces the risk of introducing errors into the production system.

1. Development Environment

The **development environment** is used by developers to implement new features, fix defects, and integrate code changes. Developers can test individual modules such as case management, document classification, hearing scheduling, AI analytics, and user authentication. Code committed to the repository can be automatically built and tested through the CI pipeline.

2. Testing/Staging Environment

The **staging environment** should closely resemble the production environment. After successful development testing, the application is automatically deployed to staging. Integration testing, API testing, security testing, performance testing, AI model validation, and end-to-end testing are performed in this environment.

Only builds that pass the required quality and security checks should be promoted to the staging environment. Test data should be **dummy, anonymized, or synthetic data** rather than confidential real court information.

3. Production Environment

The **production environment** contains the approved and stable version of the application used by authorized court personnel and other permitted users. Production deployment should occur only after successful completion of testing and appropriate approval.

Because the platform handles sensitive judicial information, production access should be protected using strong authentication, role-based access control, encryption, secure configuration, and audit logging.



Deployment Flow

Developer → Git Repository → CI Build → Automated Testing → Quality & Security Checks →
Development → Staging → Approval → Production → Monitoring

Deployment Strategy

A **Blue-Green Deployment** strategy can be used for the production environment. In this approach, the existing stable version continues to serve users while the new version is deployed separately. After successful verification, traffic can be switched to the new version. If a serious problem is detected, traffic can immediately be redirected to the previous stable version.

Alternatively, a **Rolling Deployment** can gradually replace the old application instances with the new version while maintaining service availability.

Rollback

Every production release should have a version number and a previously tested stable version should be retained. If the new release causes application errors, security problems, performance degradation, or unexpected behaviour, the system should be rolled back to the previous stable release.

Conclusion

The separation of **development, staging, and production environments** provides controlled and reliable software delivery. Automated testing and quality gates prevent unstable builds from reaching production, while blue-green or rolling deployment and rollback mechanisms reduce downtime and deployment risks. This strategy supports **reliable, secure, and continuous delivery** of the judicial platform.

Question 6: Incorporate Appropriate Security, Quality Checks, Notifications, and Rollback Mechanisms in the CI/CD Pipeline

Answer:

Security, quality assurance, notifications, and rollback mechanisms are essential components of the CI/CD pipeline for the **AI-Powered Smart Court Case Management and Judicial Analytics Platform**. Since the platform handles confidential judicial information and uses AI-based processing, the pipeline must ensure that every release is secure, reliable, and of acceptable quality.

1. Security Measures

Security checks should be integrated throughout the CI/CD pipeline rather than performed only after deployment.

- **Secure source code:** Git repositories should use proper access controls and branch protection.
- **Secret management:** Passwords, API keys, database credentials, and other secrets should not be stored directly in source code. Secure secret-management facilities should be used.

- **Dependency scanning:** Third-party libraries should be scanned for known vulnerabilities using tools such as OWASP Dependency-Check.
- **Static security analysis:** Source code should be automatically checked for common security vulnerabilities.
- **Access control:** Only authorized developers and administrators should be permitted to modify the pipeline or deploy to production.
- **Data protection:** Sensitive judicial information should be encrypted during transmission and storage.
- **Audit logging:** Code changes, approvals, deployments, and security events should be recorded for accountability.

2. Quality Checks

The pipeline should contain quality gates before allowing an application to move to the next stage.

- Build must complete successfully.
- Unit and integration tests must pass.
- API and end-to-end tests should pass.
- Code quality should meet the defined SonarQube quality threshold.
- Security scans should not identify critical unresolved vulnerabilities.
- AI/NLP model validation should meet predefined accuracy requirements.
- Performance should remain within acceptable limits.

If a critical quality or security check fails, the pipeline should automatically stop and prevent deployment.

3. Notifications

Automated notifications should inform the development and administration teams about important pipeline events.

Notifications can be generated for:

- Successful builds
- Failed builds
- Failed automated tests
- Security vulnerabilities
- Successful staging deployment
- Production deployment
- Production deployment failure
- Rollback events

Notifications can be delivered through **email or Microsoft Teams** so that responsible personnel can take immediate action.

4. Rollback Mechanism

A rollback mechanism is required to recover quickly if a production deployment causes serious problems. Before deployment, the current stable version should be retained. Each release should have a unique version number.

If the new version causes application failures, security issues, performance degradation, or unexpected behaviour, the deployment system should restore the previous stable version. With a **Blue-Green Deployment** approach, traffic can be redirected to the previous environment quickly.

CI/CD Quality and Security Flow

Code Commit → Build → Automated Tests → Code Quality → Security Scan → AI Validation → Package → Staging → Approval → Production → Monitoring → Rollback if Required

Conclusion

Integrating security, quality checks, notifications, and rollback mechanisms makes the CI/CD pipeline more **secure, reliable, and resilient**. These practices ensure that only properly tested and approved software reaches production while providing rapid recovery when unexpected problems occur. This is especially important for a judicial platform where **security, confidentiality, reliability, transparency, and accountability** are critical.

Question 7: Prepare a CI/CD Pipeline Diagram and Explain the Workflow from Code Commit to Deployment

Answer:

The CI/CD pipeline for the **AI-Powered Smart Court Case Management and Judicial Analytics Platform** automates the complete software delivery process, starting from source-code changes and ending with production deployment and monitoring.

CI/CD Pipeline Diagram

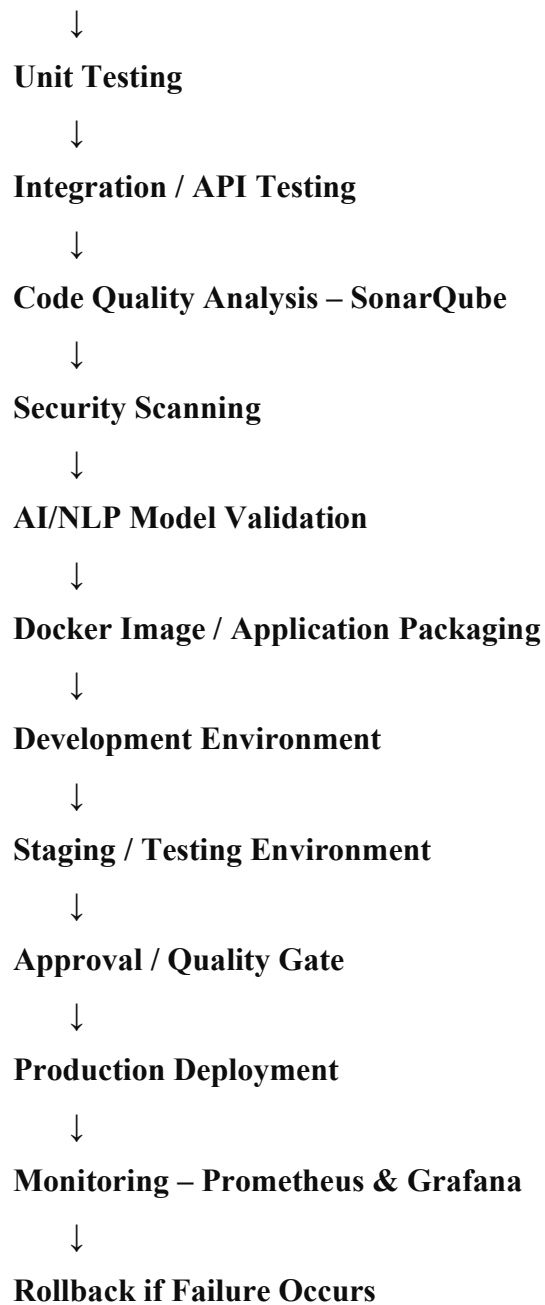
Developer



GitHub Repository



Build & Dependency Installation



Workflow Explanation

1. Code Commit:

Developers write or modify application code and commit the changes to the GitHub repository. The commit automatically triggers the CI/CD pipeline.

2. Build:

The pipeline downloads the required dependencies and builds the application. If the build fails, the pipeline stops and the development team is notified.

3. Automated Testing:

Unit, integration, API, and selected end-to-end tests are executed automatically. Only builds that pass the required tests continue to the next stage.

4. Quality and Security Analysis:

SonarQube performs code-quality analysis, while security tools scan the application and its dependencies for vulnerabilities. Critical issues prevent further deployment.

5. AI/NLP Validation:

The AI-based document classification and prediction modules are tested against predefined datasets to ensure that their performance remains within acceptable limits.

6. Packaging:

After successful validation, the application is packaged as a Docker image or another deployable artifact. A unique version number is assigned to the package.

7. Development and Staging Deployment:

The application is first deployed to the development environment and then to the staging environment. Additional testing is performed in staging using safe test data.

8. Approval and Production Deployment:

After all quality gates are passed, authorized personnel approve the release. The approved version is deployed to the production environment.

9. Monitoring:

After deployment, system health, performance, availability, and errors are continuously monitored using appropriate monitoring tools.

10. Rollback:

If a serious production issue is detected, the deployment is rolled back to the previously stable version to minimize service disruption.

Conclusion

The proposed CI/CD pipeline provides a complete automated workflow from **code commit to production deployment**. It integrates testing, code quality, security, AI validation, deployment controls, monitoring, and rollback mechanisms. This ensures that the judicial platform can be updated efficiently while maintaining **high quality, security, reliability, transparency, and operational stability**.